UNIVERSIDAD DE ANTIOQUIA
Facultad de Ingeniería

LABORATORIO ELECTRÓNICA DIGITAL III
DEPARTAMENTO DE INGENIERÍA ELECTRÓNICA Y TELECOMUNICACIONES
PRÁCTICA 5 – Medición de Frecuencia y Duty

OBJETIVOS

- Implementar y diferenciar entre los flujos de polling, IRQs y polling + IRQs en una aplicación de medición y comunicación concurrente.
- Medir la frecuencia y el ciclo de trabajo, y caracterizar los errores y el intervalo de operación alcanzable mediante pruebas reproducibles.
- Desarrollar un driver UART de transmisión mediante interrupciones, con gestión de datos pendientes y un comportamiento definido ante la falta de espacio disponible en los buffers o FIFOs de transmisión.
- Estructurar y documentar el firmware, justificar las decisiones de diseño y mantener la trazabilidad entre requisitos, implementación, pruebas y evidencias.

DESCRIPCIÓN

El equipo desarrollará un instrumento que mida la frecuencia y el ciclo de trabajo de una señal digital periódica y reporte ambas magnitudes al computador por UART. Implementará tres versiones en C y comparará su exactitud y su capacidad para manejar eventos. El desarrollo seguirá la Guía metodológica de laboratorios del curso.

FUNCIÓN DEL INSTRUMENTO

El sistema recibirá una señal rectangular periódica, medirá su frecuencia y el porcentaje del periodo en el que permanece en nivel alto, y transmitirá los resultados mientras continúa la medición. El término utilizado en esta guía es "ciclo de trabajo" o "duty cycle", que corresponde al ciclo de dureza mencionado en el enunciado original.

Si T es el periodo y tH es el tiempo en nivel alto, las magnitudes son $f = \frac{1}{T}$ y $D = 100 \times \frac{tH}{T}$. T y tH se expresan en segundos, f en hertz y D en porcentaje. La frecuencia de la señal de entrada es independiente de la frecuencia con la que se envían los reportes.

La entrada y el enlace UART usarán niveles lógicos compatibles con 3,3 V y una referencia de tierra común. El equipo verificará los límites eléctricos del montaje antes de conectar la fuente. El enlace al computador utilizará una UART física del RP2040 y un adaptador USB a UART compatible. El USB nativo de la Pico podrá usarse para programación; no sustituye la UART evaluada.

La salida del sistema se mostrará en una terminal serial. Se exige únicamente la transmisión; no se solicita un driver de recepción.

CONDICIONES COMUNES DE FUNCIONAMIENTO

El siguiente perfil establece las condiciones de comparación para las tres versiones. El límite superior alcanzable será un resultado experimental; no se exige medir señales de cualquier frecuencia.

Magnitud o condición	**Especificación común**
Perfil nominal	Frecuencia de entrada entre 20 Hz y 10 kHz; ciclo de trabajo entre 5 % y 95 %.
Error de frecuencia	Error relativo absoluto no mayor que 1 % respecto a la referencia medida.
Error de ciclo de trabajo	Diferencia absoluta no mayor a un punto porcentual respecto a la referencia medida.
Reporte nominal	10 reportes por segundo; periodo de 100 ms con desviación máxima de ±20 ms, observada en la salida UART.

Magnitud o condición	Especificación común
Cambio de entrada	Tras un cambio entre condiciones nominales, reportar valores dentro de la tolerancia en un máximo de 500 ms.
Entrada sin transiciones	Invalidar la medición después de 500 ms si no se detectan transiciones y comunicarlo en el siguiente reporte programado.

Los errores se verifican en los valores transmitidos, incluido el redondeo. Para la frecuencia, se utiliza

$$E_f = 100 \times \frac{abs(f_{medida} - f_{referencia})}{f_{referencia}}$$

Para el ciclo de trabajo, se utiliza

$$E_d = abs(D_{medido} - D_{referencia})$$

Con D_{medido} y $D_{referencia}$ en puntos porcentuales. La cantidad de decimales del reporte no refleja su exactitud.

Antes de disponer de una medición completa, y cuando no se detecten transiciones durante el plazo indicado, el sistema reportará valores no disponibles. Una entrada constante no permite inferir simultáneamente un periodo y un ciclo de trabajo; no debe presentarse una medición anterior como vigente. El comportamiento fuera del intervalo caracterizado se documentará, sin exigir la detección de todos los casos de pérdida de eventos.

Reporte por UART

La UART operará a 115200 bits/s, con 8 bits de datos, sin paridad, 1 bit de parada y sin control de flujo. Cada reporte será una línea ASCII de hasta 64 bytes, terminada en LF, con cuatro campos separados por comas:

$$\boldsymbol{n,\Delta t,f_{Hz},D_{pct}}\text{,estado}$$

Por ejemplo, **17,100,1000000,25.0,OK** representa una señal de 1 kHz con un ciclo de trabajo del 25 %. Un reporte sin medición válida será **18,101,NA,NA,SIN_TRANSICIONES**. Durante la espera inicial, se utilizará el estado ESPERANDO. Los decimales usan punto; la frecuencia se expresa con tres decimales y el ciclo de trabajo con uno. No se incluyen unidades ni mensajes de depuración en las líneas de datos. Δt Es el tiempo, en milisegundos, transcurrido desde la medición anterior hasta la actual que se reporta. El Δt en el primer reporte siempre será cero.

El campo n es un entero sin signo de 32 bits que comienza en cero al arrancar y avanza con cada reporte programado, aunque no se envíe. Permite identificar omisiones en el archivo recibido. Las líneas no se entrelazan ni se truncan por falta de espacio; los descartes ocurren antes de comenzar una línea. El equipo contabilizará los reportes omitidos o descartados y documentará cómo consulta dicho conteo al finalizar la prueba.

Tres versiones del firmware

Versión	Organización durante la operación
Polling	El ciclo principal consulta la entrada, los tiempos y la UART, y realiza la medición, el procesamiento y el envío. No utiliza interrupciones de aplicación para esas funciones. Se permite polling sin bloqueo.
IRQs	La atención de la entrada, el cálculo, la preparación de reportes y el servicio UART se ejecutan en contextos de interrupción. El ciclo principal no despacha trabajo de la aplicación.

Versión	Organización durante la operación
Polling+IRQs	Las ISR atienden el hardware y conservan información o notifican trabajo pendiente. El ciclo principal consulta ese trabajo y realiza el cálculo y la preparación de los reportes. El servicio de transmisión UART opera mediante interrupciones.

Las tres versiones usarán C, Visual Studio Code y el SDK oficial de Raspberry Pi Pico. Podrán compartir módulos cuando su uso respete el flujo correspondiente. La inicialización podrá ejecutarse fuera de las ISR.

Para aislar el efecto del flujo, la aplicación usará un solo núcleo en las tres versiones. Se conservarán el montaje, la frecuencia de reloj, las opciones de compilación y las condiciones de prueba. No se utilizarán DMA, PIO ni un periférico externo para medir la señal de forma autónoma, en lugar de hacerlo con los recursos del MCU. Se permite usar temporizadores como referencia temporal y para generar eventos de la aplicación.

Se permiten el SDK oficial y la biblioteca estándar de C; no se utilizarán bibliotecas de terceros para la medición, la gestión del flujo o el driver UART. El método de medición, la representación numérica, la organización de módulos, las prioridades, los buffers y la sincronización serán decisiones del equipo. Las condiciones pedagógicas anteriores son entradas al diseño y no requieren inventar alternativas para justificarlas.

Driver UART propio

En IRQs y Polling+IRQs, el equipo desarrollará el driver de transmisión UART. Podrá reutilizarlo entre ambas versiones. Deberá implementar el servicio por interrupciones, conservar los datos hasta su transferencia, gestionar la activación del servicio y definir la respuesta ante la falta de espacio. Habilitar una interrupción sin implementar el servicio no cumple con esta exigencia.

Se permite el uso del SDK para la inicialización, la definición de registros y la instalación de manejadores. Los demás accesos al hardware deben ser coherentes con el flujo declarado. Dentro de una ISR se puede consultar el estado para transferir lo que ya cabe en el hardware; no se permite esperar a que aparezca espacio nuevo.

El método *uart_write_blocking* puede detener la ejecución hasta entregar todos los bytes al hardware, y *uart_tx_wait_blocking* espera a que finalice la transmisión [2]. Esa espera ocupa el contexto de ejecución y puede retrasar otras actividades. Estas funciones no proporcionan, por sí solas, el driver IRQ con los buffers requeridos.

En las dos versiones con interrupciones no se usarán *printf* ni las funciones de salida de *stdio* para transmitir los reportes, ni rutinas que esperen activamente espacio en la UART. Se permite formatear texto en memoria, por ejemplo, con *snprintf*, y entregarlo al driver propio. En polling se admite el soporte de envío por polling del SDK, cuyo efecto deberá evaluarse; no se exige introducir bloqueos artificiales.

Procedimiento

Interpretación y requisitos

Aplicar las seis fases de la guía metodológica, sin repetir sus definiciones en el informe. Delimitar entradas, salidas, estados y restricciones; formular requisitos funcionales, temporales y de interfaz, con identificadores únicos, fuente y criterio de verificación. Separar las condiciones docentes de las especificaciones particulares del montaje. Establecer los criterios antes de implementar y registrar cualquier revisión justificada.

Planificación de pruebas y diseño

Definir pruebas de componentes, de integración y de aceptación para los requisitos. Cada prueba indicará las condiciones iniciales, los estímulos, los instrumentos, las magnitudes, las tolerancias y la evidencia esperada. Preparar la matriz de requisito–decisión–prueba–evidencia.

Comparar alternativas para las decisiones relevantes que aún están abiertas. Documentar el contexto, las opciones viables, los criterios, la evidencia, la selección y las consecuencias. Definir responsabilidades e interfaces entre medición, procesamiento y comunicación; representar el flujo y los contextos de ejecución de cada versión. Analizar la resolución temporal, el tiempo de cómputo y la capacidad de transmisión antes de integrar.

Implementación e integración

Verificar por separado la entrada, las mediciones y el servicio UART. En el driver propio, probar envíos de distintas longitudes, con datos pendientes y con falta de espacio. Integrar gradualmente hasta obtener las tres versiones funcionalmente equivalentes. Documentar el código con Doxygen, incluyendo unidades, interfaces, contexto de ejecución, propiedad de los buffers y manejo de errores. Conservar un registro breve de fallos relevantes y de correcciones.

Verificación funcional

Usar una fuente de señal independiente del firmware evaluado, como un generador de laboratorio. Medir la señal con un instrumento de referencia; no asumir que el valor programado en el generador coincide exactamente con el aplicado. Registrar los niveles, la frecuencia, el ciclo de trabajo y las limitaciones del instrumento. No es necesario adquirir otro microcontrolador.

Comparación de desempeño

Límite de medición. Para cada versión y para los ciclos de trabajo del 10 %, 50 % y 90 %, aumentar progresivamente la frecuencia de entrada a partir de una condición válida. Registrar al menos 10 s por punto. Al encontrar el primer incumplimiento, acotar la última condición válida y la primera fallida, con una separación de frecuencia no mayor al 10 % de la última válida. Repetir ambas condiciones para comprobar que el resultado es reproducible.

Un punto solo cumple si satisface simultáneamente las tolerancias de frecuencia y de ciclo de trabajo, y no presenta reportes omitidos, líneas corruptas ni incumplimiento del periodo de reporte. Informar el límite por ciclo de trabajo y el menor de esos límites. Si el instrumento no permite alcanzar el fallo, reportar la mayor frecuencia verificada; no presentarla como límite del firmware. Un resultado correcto aislado, aunque frecuente, no demuestra un intervalo de operación continuo.

Efecto de la comunicación. Seleccionar tres frecuencias comunes a las tres versiones: una baja, una intermedia y otra cercana al límite superior observado. A un 50 % de ciclo de trabajo, incrementar progresivamente el número de reportes por segundo y repetir cada condición durante al menos 10 s. Comparar el error, la dispersión, la frecuencia de reporte alcanzada y las omisiones.

Conservar archivos de terminal, tablas y capturas de instrumentos. La regularidad temporal observada en el PC incluye efectos del adaptador y del sistema operativo; verificar el periodo de la señal TX al evaluar el cumplimiento temporal. Identificar las causas de los fallos sin exigir que una modalidad resulte siempre superior.

Informe y entregables

Entregar un único informe con alcance y requisitos; pruebas y trazabilidad; decisiones de diseño; esquemas e interfaces; organización de las tres versiones; evidencias de integración; resultados comparativos y conclusiones. Las tablas de comparación deben identificar el ciclo de trabajo, la frecuencia de entrada, la tasa de reporte, los errores, las omisiones y la versión evaluada. Vincular cada evidencia con su prueba y su requisito.

Adjuntar el código fuente en C y los archivos de construcción de las tres versiones, la documentación de Doxygen, las instrucciones para compilar y ejecutar, la asignación de pines y las evidencias de las pruebas. Indicar la versión del SDK, la frecuencia de reloj y las opciones de compilación.

Subir el informe, el código y las evidencias en un archivo comprimido a Moodle, identificado con los apellidos de los integrantes. La fecha y la hora de entrega serán las que consten en la actividad de Moodle.

Evaluación

Cada criterio se califica de 0 a 5 y se pondera según el porcentaje indicado. La nota final es la suma de las calificaciones ponderadas, en una escala de 0 a 5. Los niveles orientan el juicio con base en la evidencia; se permiten calificaciones intermedias justificadas.

Criterio	Peso	Evidencia para alcanzar el nivel completo
Versión polling	10 %	Mide ambas magnitudes, reporta en el perfil nominal y respeta el flujo de consultas; verifica los cambios de entrada y la ausencia de transiciones.
Versión de interrupciones puras	15 %	Cumple las funciones nominales y ejecuta el trabajo de aplicación en ISR; demuestra un driver UART propio sin espera activa.
Versión polling + interrupciones	30 %	Cumple las funciones nominales; distingue entre atención al hardware y procesamiento diferido; demuestra un driver UART propio sin espera activa.
Comparación experimental	15 %	Caracteriza los límites de la medición y la reproducibilidad de los resultados, y explica los fallos.
Aplicación de la metodología	20 %	Requisitos verificables, pruebas previas, decisiones justificadas, integración incremental y trazabilidad completa hasta la evidencia.
Código y documentación	5 %	Módulos e interfaces claros, manejo de errores, documentación en Doxygen e instrucciones de construcción reproducibles.
Sustentación	5 %	Explica el código propio, los contextos de ejecución, las decisiones tomadas y las limitaciones de los resultados.
Total	100 %	

Nivel completo de 4,0 a 5,0. Cumple el criterio con evidencia suficiente y coherente; las limitaciones se identifican y se explican. *Nivel parcial de 2,0 a 3,9*. Presenta avances verificables, pero existen incumplimientos, errores de interpretación o evidencia incompleta. *Nivel insuficiente de 0,0 a 1,9*. El resultado falta, no puede reproducirse o contradice la organización exigida.

La ausencia de una versión se califica con cero según su criterio. Si el equipo no implementa el driver UART de transmisión propio en cualquiera de las dos versiones que lo requieren, la nota final máxima será de 2,5 sobre 5,0 tras aplicar la ponderación. Reutilizar el mismo driver propio en ambas versiones está permitido.

La frecuencia máxima alcanzada no se califica mediante una comparación competitiva entre equipos. Se evalúan el cumplimiento nominal, la corrección de los flujos, la calidad del experimento y el análisis de sus límites, debidamente sustentado.